

Intensivtutorium

Programmierung

Semester: Sommersemester 2026

Tutor: Luis Staudt

Studiengänge: MIB, OMB, PEB, MVB

Aufgabe 1 – Kinoreservierung

Ein kleines Kino verwaltet seine Sitzplätze mit einem zweidimensionalen Array. Jeder Eintrag steht für einen Sitzplatz: `true` bedeutet *belegt*, `false` bedeutet *frei*.

```
1 boolean[][] kino = new boolean[reihen][plaetze];
```

Der Saal hat `reihen` Reihen mit jeweils `plaetze` Sitzen. Zu Beginn sind alle Plätze frei.

	p →							
	0	1	2	3	4	5	6	7
Reihe 0:	0	0	X	0	0	0	0	0
Reihe 1:	0	0	0	0	0	0	0	0
Reihe 2:	0	X	X	X	0	0	0	0
Reihe 3:	0	0	0	0	0	0	0	0
Reihe 4:	0	0	0	0	0	X	X	0

0 = frei
X = belegt

Hinweis

Der Index `kino[r][p]` steht für Reihe `r`, Platz `p` (jeweils ab 0).

Implementiere die folgenden Methoden:

- a) **Saal anzeigen** – Gibt den Saal zeilenweise auf der Konsole aus. Verwende `X` für belegte und `0` für freie Plätze. Vor jeder Zeile soll die Reihennummer stehen.

```
1 static void anzeigen(boolean[][] kino)
```

- b) **Platz reservieren** – Reserviert den Platz in Reihe `r`, Position `p`. Ist der Platz bereits belegt oder sind die Indizes ungültig, soll eine Fehlermeldung auf der Konsole ausgegeben und `false` zurückgegeben werden. Bei erfolgreicher Reservierung wird `true` zurückgegeben.

```
1 static boolean reservieren(boolean[][] kino, int r, int p)
```

c) **Freie Plätze zählen** – Gibt die Gesamtanzahl aller freien Plätze im Kino zurück.

```
1 static int freiePlaetze(boolean[][] kino)
```

d) **Reihe komplett frei?** – Prüft, ob in der angegebenen Reihe *alle* Plätze frei sind.

```
1 static boolean reiheFrei(boolean[][] kino, int reihe)
```

e) **Gruppenreservierung** – Eine Gruppe von *anzahl* Personen möchte nebeneinander in *derselben Reihe* sitzen. Die Methode sucht den ersten zusammenhängenden freien Bereich, der groß genug ist, reserviert die Plätze und gibt die Reihe zurück. Wird kein passender Bereich gefunden, wird *-1* zurückgegeben.

```
1 static int gruppenReservierung(boolean[][] kino, int anzahl)
```

Hinweis

Für Teilaufgabe e) muss in jeder Reihe nach einem zusammenhängenden Block aus *anzahl* freien Plätzen gesucht werden. Überlege dir, wie du die Länge des aktuellen freien Bereichs beim Durchlaufen mitzählen kannst.

f) **Belegungsgrad** – Berechnet den prozentualen Anteil der belegten Plätze und gibt ihn als *double* zurück (z. B. 42.5 für 42,5 %).

```
1 static double belegungsgrad(boolean[][] kino)
```